TA8: An Example of GLM in STAN

06/11/2026

```
# Import modules
library(posterior)
library(bayesplot)
library(ggplot2)

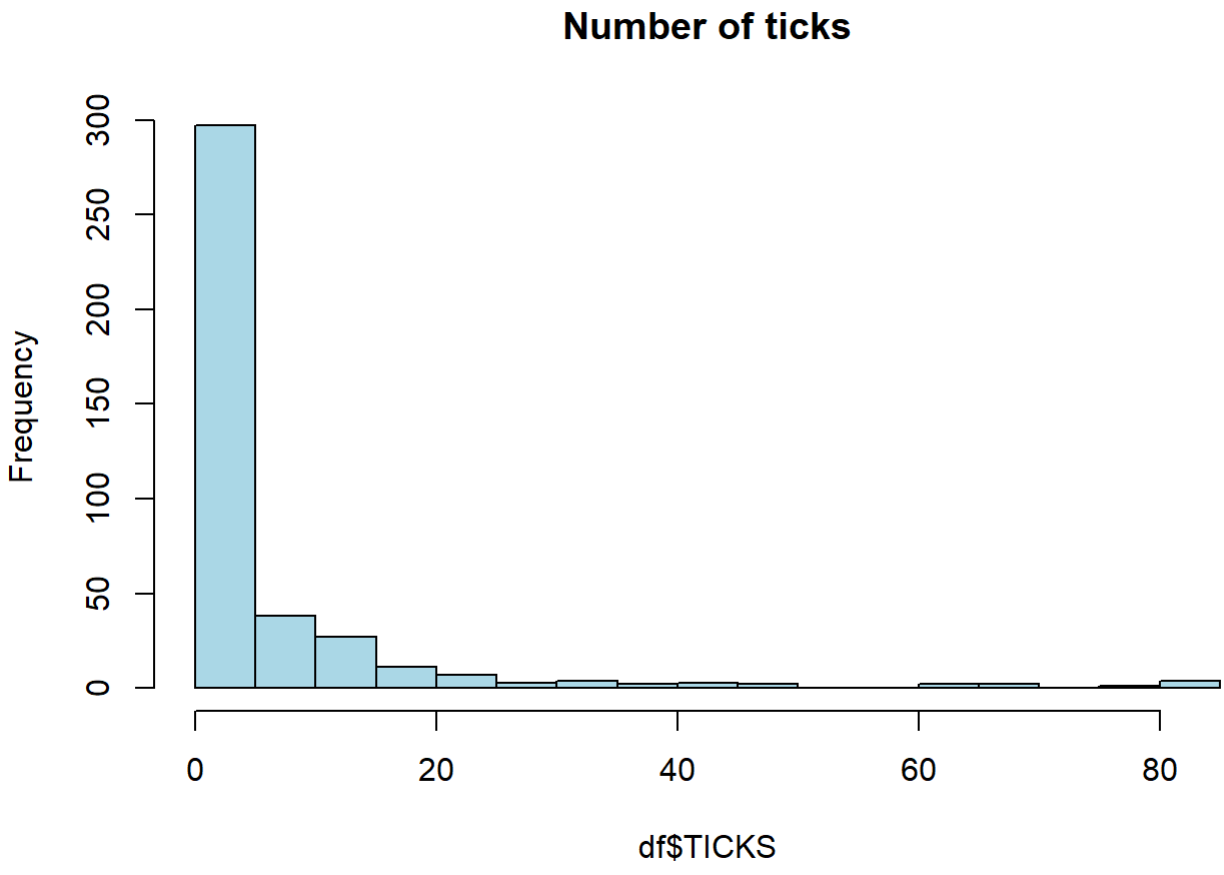
# Import functions
library(cmdstanr)

# Create ./stan folder if does not exists
if (!dir.exists("./stan")) {
    dir.create("./stan")
}
```

```
df <- read.csv("grouse_data.csv", row.names = 1)
head(df)
```

```
##   TICKS HEIGHT YEAR LOCATION
## 1     0    465   95       32
## 2     0    465   95       32
## 3     0    472   95       36
## 4     0    475   95       37
## 5     0    475   95       37
## 6     3    475   95       37
```

```
hist(df$TICKS, breaks = 20, main = 'Number of ticks', col = 'lightblue')
```



Let y_i number of ticks on a grouse. As covariates we consider $x_i = [1, h_i]$ where h_i is the height of the place where the grouse lives.

As standard practice we standardize the covariates

The model assumes

$$\begin{aligned} y_i \mid \beta &\sim \text{Poi}(\exp(x_i^T \beta)) \\ \beta_j \mid \sigma_j &\sim \mathcal{N}(0, \sigma_j^2) \\ \sigma_j &\sim \mathcal{HC}(0, 1) \end{aligned}$$

Where $\mathcal{HC}(0, 1)$ is the half-Cauchy distribution (i.e. Cauchy truncated on $[0, +\infty)$ centered in 0 and with scale 1.

```
y <- df$TICKS
h <- df$HEIGHT
h <- (h - mean(h)) / sd(h)

X <- cbind(1, h)
X[1:10, ]
```

```
##                 h
##  [1,] 1 0.0767401
##  [2,] 1 0.0767401
##  [3,] 1 0.2714198
##  [4,] 1 0.3548539
##  [5,] 1 0.3548539
##  [6,] 1 0.3548539
##  [7,] 1 0.3548539
##  [8,] 1 0.7164019
##  [9,] 1 0.7164019
## [10,] 1 0.7164019
```

```
poi_glm <- "
data {
    int<lower=0> N;
    int<lower=0> p;
    array[N] int<lower=0> Y;
    matrix[N, p] X;
}

parameters {
    vector[p] beta;
    vector<lower=0>[p] sigma_beta;
}


transformed parameters {
    vector[N] mu;
    for(i in 1:N) {
      mu[i] = exp(row(X, i) * beta);
    }
}

model {
    for (s in 1:N) {
        Y[s] ~ poisson(mu[s]);
    }


    for (j in 1:p) {
        beta[j] ~ normal(0.0, sigma_beta[j]);
        sigma_beta[j] ~ cauchy(0.0, 1.0);
    }
}

generated quantities {
  vector[N] log_lik;
  for (j in 1:N) {
    log_lik[j] = poisson_lpmf(Y[j] | mu[j]);
  }
}
"

# Write stan model to file
stan_file <- "./stan/poisson_glm.stan"
writeLines(poi_glm, stan_file)

# Compile stan model
poi_glm <- cmdstan_model(stan_file)
```

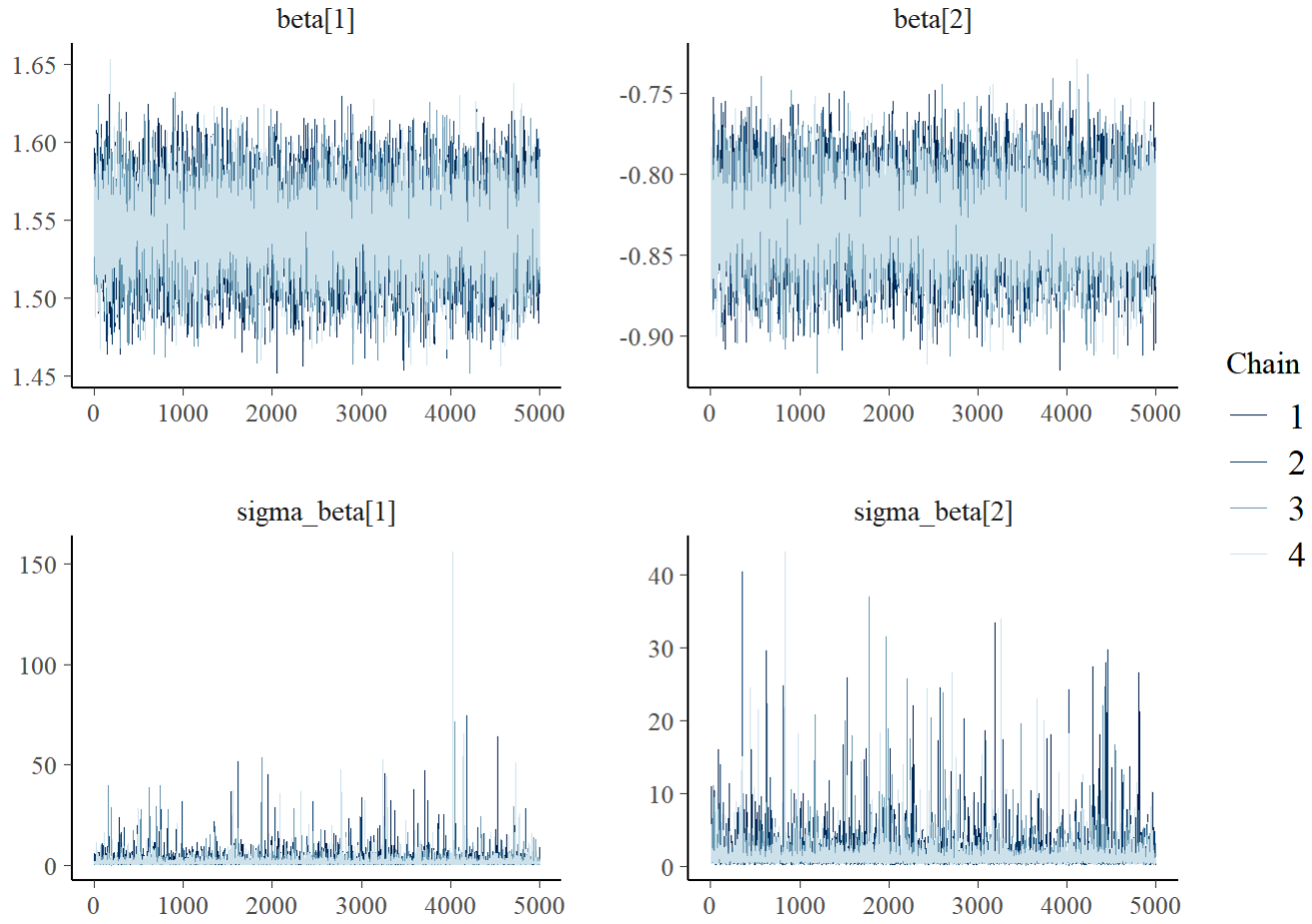
```
# Input data
glm_data <- list(
    N = length(y),
    p = ncol(X),
    Y = y,
    X = X
)

# Sample
poi_glm_fit <- poi_glm$sample(
    data = glm_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 5000,
    refresh = 2000
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 1 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 2 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 3 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 4 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 1 finished in 4.1 seconds.
## Chain 2 finished in 4.1 seconds.
## Chain 3 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 3 finished in 4.2 seconds.
## Chain 4 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 4 finished in 0.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 3.1 seconds.
## Total execution time: 4.4 seconds.
```

```
# Convert to arviz data type
poi_glm_data <- posterior::as_draws(poi_glm_fit)
```

```
mcmc_trace(poi_glm_data, pars = c("beta[1]", "beta[2]", "sigma_beta[1]", "sigma_beta[2]"))
```




```
sum(poi_glm_fit$sampler_diagnostics()[, , "divergent__"])
```

```
## [1] 0
```

Incorporating additional information in the model

We have that data are divided by years (95, 96, 97).

How can we take this into account?

Of course adding the year as a linear predictor makes very little sense

Mixed models

We consider that there is a “random effect” specific for each year:

$$
\begin{aligned}
y_{i,t} \mid \mu_{i,t} &\sim \text{Poi}(\mu_{i,t}) \\
\mu_{i,t} &= \exp\left(x_i^T \beta + \lambda_t\right) \\
\beta_j \mid \sigma_j &\sim \mathcal{N}(0, \sigma_j^2) \\
\sigma_j &\sim \mathcal{HC}(0, 1) \\
\lambda_1, \ldots, \lambda_T \mid \lambda_0, s_0 &\sim \mathcal{N}(\lambda_0, s_0) \\
\lambda_0, s_0 &\sim \mathcal{N}(\lambda_{00}, \tau_0) \times \mathcal{HC}(0, 1)
\end{aligned}
$$

Where $y_{i,t}$ makes the dependence on the year explicit (t).

Here λ_t is a random intercept specific of the year

A more convenient formulation is

$$
\begin{aligned}
y_i \mid \mu_i &\sim \text{Poi}(\mu_i) \\
\mu_i &= \exp\left(x_i^T \beta + g_i^T \lambda\right) \\
\beta_j \mid \sigma_j &\sim \mathcal{N}(0, \sigma_j^2) \\
\sigma_j &\sim \mathcal{HC}(0, 1) \\
\lambda_1, \ldots, \lambda_T \mid \lambda_0, s_0 &\sim \mathcal{N}(\lambda_0, s_0) \\
\lambda_0, s_0 &\sim \mathcal{N}(\lambda_{00}, \tau_0) \times \mathcal{HN}(0, 3)
\end{aligned}
$$

where $\lambda = (\lambda_1, \lambda_2, \lambda_3)^T$ and g_i is an indicator vector (i.e. $g_i[h] = 1$ iff observation i belongs to group h)

We fix $\lambda_{00} = 0$, $\tau_0 = 3$.

```
poi_glmm_grouse <- "
data {
    int<lower = 0> N;
    int<lower = 0> p_fix;
    int<lower = 0> n_groups;

    array[N] int<lower=0> Y;
    matrix[N, p_fix] X;
    matrix[N, n_groups] G;
}

parameters {
    vector[p_fix] beta;
    vector<lower=0>[p_fix] sigma_beta;

    vector[n_groups] lambda_raw;
    real lambda0;
    real<lower=0> s0;
}

transformed parameters  {
    vector[n_groups] lambda;
    vector[N] mu;

    lambda = lambda0 + lambda_raw * s0;

    mu = exp(X * beta + G * lambda);
}

model {
    for (s in 1:N) {
        Y[s] ~ poisson(mu[s]);
    }

    for (j in 1:p_fix) {
        beta[j] ~ normal(0.0, sigma_beta[j]);
        sigma_beta[j] ~ cauchy(0.0, 1.0);
    }

    lambda_raw ~ normal(0.0, 1.0);
    lambda0 ~ normal(0.0, 3.0);
    s0 ~ cauchy(0.0, 3.0);
}

generated quantities  {
    vector[N] log_lik;
    for (j in 1:N) {
        log_lik[j] = poisson_lpmf(Y[j] | mu[j]);
    }
}
"

stan_file <- "./stan/poi_glmm_grouse.stan"
writeLines(poi_glmm_grouse, stan_file)

poi_glmm <- cmdstan_model(stan_file)
```

```
years <- df$YEAR
G <- outer(years, unique(years), "==") * 1
```

```
# Drop the intercept due to the presence of random effects with mean != 0
X <- X[, 2, drop = FALSE]
```

```
glmm_data <- list(
    N = length(y),
    p_fix = ncol(X),
    n_groups = ncol(G),
    Y = y,
    X = X,
    G = G
)

poi_glmm_fit <- poi_glmm$sample(
    data = glmm_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 1000,
    refresh = 500
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of th
e following issue:
```

```
## Chain 1 Exception: poisson_lpmf: Rate parameter is nan, but must be nonnegative! (in 'C:/Users/gbel
t/AppData/Local/Temp/Rtmpwzfbmt/model-55d0456f19b1.stan', line 32, column 8 to column 30)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like cova
riance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
isspecified.
```

```
## Chain 1
```

```
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of th
e following issue:
```

```
## Chain 3 Exception: poisson_lpmf: Rate parameter is nan, but must be nonnegative! (in 'C:/Users/gbel
t/AppData/Local/Temp/Rtmpwzfbmt/model-55d0456f19b1.stan', line 32, column 8 to column 30)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like cova
riance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
isspecified.
```

```
## Chain 3
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of th
e following issue:
```

```
## Chain 3 Exception: poisson_lpmf: Rate parameter is nan, but must be nonnegative! (in 'C:/Users/gbel
t/AppData/Local/Temp/Rtmpwzfbmt/model-55d0456f19b1.stan', line 32, column 8 to column 30)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like cova
riance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
isspecified.
```

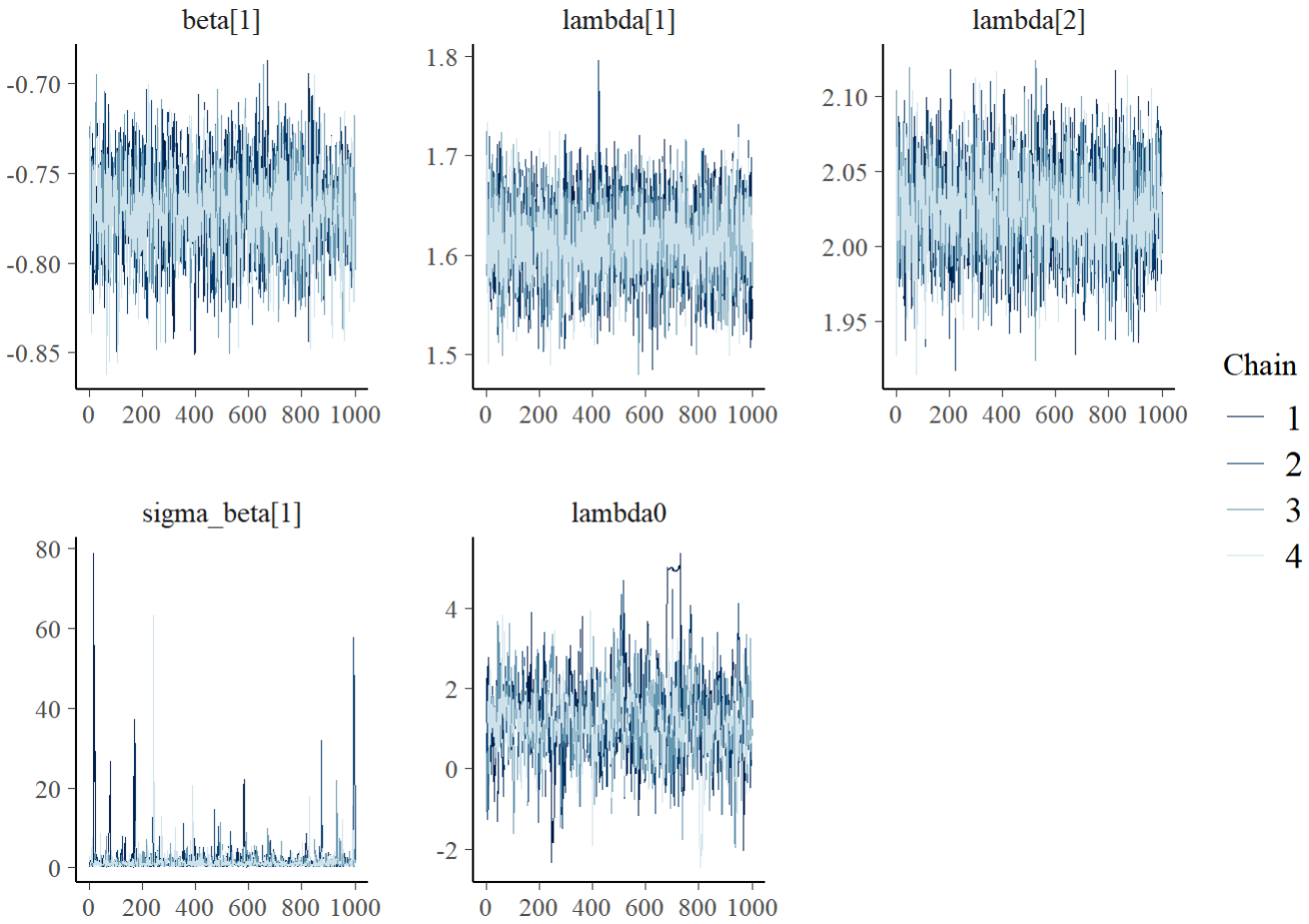
```
## Chain 3
```

```
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 24.1 seconds.
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 24.4 seconds.
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 25.6 seconds.
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 26.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 25.1 seconds.
## Total execution time: 26.4 seconds.
```

```
## Warning: 54 of 4000 (1.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.
```

```
poi_glmm_data <- posterior::as_draws(poi_glmm_fit)
```

```
mcmc_trace(poi_glmm_data, pars = c("beta[1]", "lambda[1]", "lambda[2]", "sigma_beta[1]", "lambda0"))
```



Oh no, some divergent iterations! Let's try and increase `adapt_delta` and/or the `max_treedepth`

```
poi_glmm_fit <- poi_glmm$sample(
    data = glmm_data,
    chains = 4,
    parallel_chains = 4,
    adapt_delta = 0.99,
    max_treedepth = 15,
    refresh = 500,
    seed = 345
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th
e following issue:
```

```
## Chain 2 Exception: poisson_lpmf: Rate parameter is nan, but must be nonnegative! (in 'C:/Users/gbel
t/AppData/Local/Temp/Rtmpwzfbmt/model-55d0456f19b1.stan', line 32, column 8 to column 30)
```

```
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like cova
riance matrices, then the sampler is fine,
```

```
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
isspecified.
```

```
## Chain 2
```

```
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of th
e following issue:
```

```
## Chain 3 Exception: poisson_lpmf: Rate parameter is nan, but must be nonnegative! (in 'C:/Users/gbel
t/AppData/Local/Temp/Rtmpwzfbmt/model-55d0456f19b1.stan', line 32, column 8 to column 30)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like cova
riance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
isspecified.
```

```
## Chain 3
```

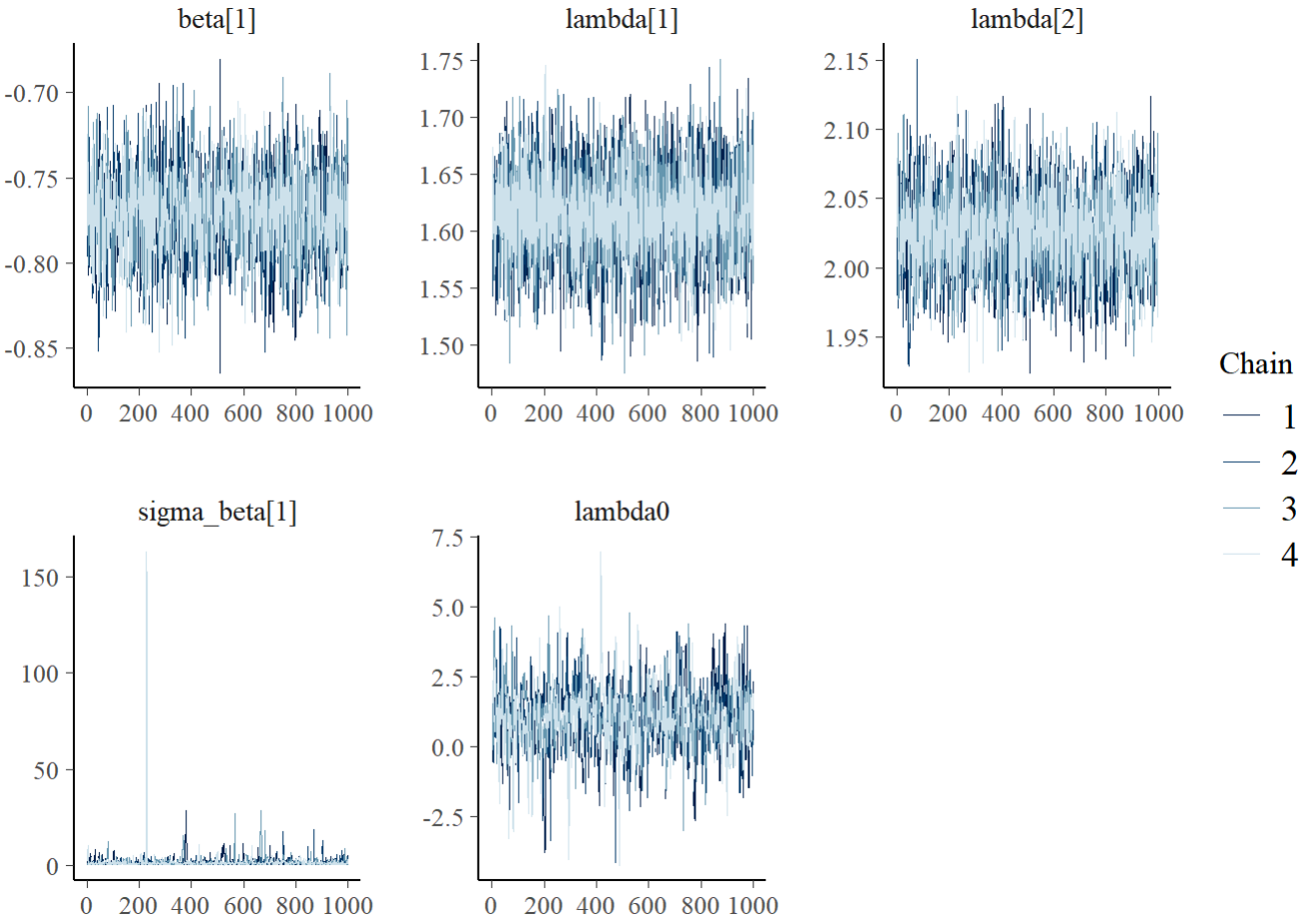
```
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 56.9 seconds.
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 72.3 seconds.
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 72.7 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 75.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 69.4 seconds.
## Total execution time: 75.9 seconds.
```

```
poi_glmm_data <- posterior::as_draws(poi_glmm_fit)
```

```
poi_glmm_fit
```

```
##       variable    mean  median   sd  mad      q5     q95 rhat ess_bulk ess_tail
##  lp__          3383.02 3383.36 2.02 1.93 3379.21 3385.69 1.00     1154     1598
##  beta[1]         -0.77   -0.77 0.03 0.03   -0.81   -0.73 1.00     2541     2250
##  sigma_beta[1]    1.53    1.01 3.85 0.61    0.42    3.68 1.00     2244     1304
##  lambda_raw[1]    0.37    0.37 0.57 0.58   -0.56    1.31 1.00      998     1493
##  lambda_raw[2]    0.67    0.66 0.62 0.62   -0.34    1.71 1.00      982     1326
##  lambda_raw[3]   -0.85   -0.82 0.73 0.72   -2.15    0.25 1.00     1068     1341
##  lambda0          1.07    1.08 1.07 0.82   -0.65    2.75 1.00     1037     1267
##  s0               1.79    1.51 1.13 0.74    0.72    3.94 1.00     1006     1608
##  lambda[1]        1.61    1.62 0.04 0.04    1.55    1.68 1.00     4061     3389
##  lambda[2]        2.03    2.03 0.03 0.03    1.97    2.08 1.00     3520     3300
##
##  # showing 10 of 817 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

```
mcmc_trace(poi_glmm_data, pars = c("beta[1]", "lambda[1]", "lambda[2]", "sigma_beta[1]", "lambda0"))
```




```
sum(poi_glmm_fit$sampler_diagnostics()[, , "divergent__"])
```

```
## [1] 0
```

Which model achieves the best fit? - WAIC and PSIS-LOO-CV

Both definitions of *WAIC* and *PSIS-LOO-CV* comes from Vehtari et al. (2016) (https://link.springer.com/article/10.1007/s11222-016-9696-4).

```
library(loo)
```

```
# Compute and print GLM WAIC
log_lik_glm <- poi_glm_fit$draws("log_lik")
print(waic(log_lik_glm))
```

```
##
## Computed from 20000 by 403 log-likelihood matrix.
##
##           Estimate    SE
## elpd_waic  -2739.0 298.5
## p_waic        38.0   7.3
## waic        5477.9 596.9
##
## 17 (4.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```
# Compute and print GLMM WAIC
log_lik_glmm <- poi_glmm_fit$draws("log_lik")
print(waic(log_lik_glmm))
```

```
##
## Computed from 4000 by 403 log-likelihood matrix.
##
##           Estimate    SE
## elpd_waic  -2222.9 247.4
## p_waic        65.0  15.9
## waic        4445.7 494.8
##
## 21 (5.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```
print(loo(log_lik_glm))
```

```
##
## Computed from 20000 by 403 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -2738.8 298.4
## p_loo        37.8   7.2
## looic      5477.6 596.8
## ------
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

```
print(loo(log_lik_glmm))
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnostic') for details.
```

```
##
## Computed from 4000 by 403 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -2221.8 247.0
## p_loo        63.9  15.3
## looic      4443.5 494.0
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## Pareto k diagnostic values:
##                          Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     400   99.3%   117
##    (0.7, 1]   (bad)        1    0.2%   <NA>
##    (1, Inf)   (very bad)   2    0.5%   <NA>
## See help('pareto-k-diagnostic') for details.
```

What is the predictive distribution?

Suppose we have a new covariate x^*, what is the distribution of the associated y^*?

1. For the GLM:

$$\begin{aligned} y_i \mid \beta &\overset{iid}{\sim} \text{Poi}(\exp(x_i^T \beta)) \\ \beta_j \mid \sigma_j &\sim \mathcal{N}(0, \sigma_j^2) \\ \sigma_j &\sim \mathcal{HC}(0, 1) \end{aligned}$$

$$\begin{aligned} \mathcal{L}(y^* \mid \mathbf{y}) &= \int \mathcal{L}(y^* \mid \beta, \mathbf{y}) \mathcal{L}(\beta \mid \mathbf{y}) \text{d}\beta \\ &\approx \frac{1}{M} \sum_{j=1}^{M} \text{Poi}(\exp(x^* \, \beta^{(j)}) \end{aligned}$$

2. For the GLMM, when the observation belongs to one of the observed groups

$$\begin{aligned} y_{i,t} \mid \lambda_t, \beta &\sim \text{Poi}(\exp\left(x_i^T \beta + \lambda_t\right)) \\ \beta_j \mid \sigma_j &\sim \mathcal{N}(0, \sigma_j^2) \\ \sigma_j &\sim \mathcal{HC}(0, 1) \\ \lambda_1, \dots, \lambda_T \mid \lambda_0, s_0 &\sim \mathcal{N}(\lambda_0, s_0) \\ \lambda_0, s_0 &\sim \mathcal{N}(\lambda_{00}, \tau_0) \times \mathcal{HN}(0, 3) \end{aligned}$$

$$\begin{aligned} \mathcal{L}(y^* \mid \mathbf{y}) &= \int \mathcal{L}(y^* \mid \lambda_t, \beta, \mathbf{y}) \mathcal{L}(\lambda_t, \beta \mid \mathbf{y}) \text{d}\beta \\ &\approx \frac{1}{M} \sum_{j=1}^{M} \text{Poi}(\exp\left(x^* \beta^{(j)} + \lambda_t^{(j)}\right)) \end{aligned}$$

3. For the GLMM, when the observation belongs to a new group! We need to consider also the law of λ^*

$$\begin{aligned} \mathcal{L}(y^*, \lambda^* \mid \mathbf{y}) &= \int \mathcal{L}(y^*, \lambda^* \mid \beta, \{\lambda_j\}, \lambda_0, s_0, \mathbf{y}) \mathcal{L}(d\beta, \{d\lambda_j\}, d\lambda_0, ds_0 \mid \mathbf{y}) \\ &= \int \mathcal{L}(y^* \mid \lambda^*, \beta) \mathcal{L}(\lambda^* \mid \beta, \{\lambda_j\}, \lambda_0, s_0, \mathbf{y}) \mathcal{L}(d\beta, \{d\lambda_j\}, d\lambda_0, ds_0 \mid \mathbf{y}) \\ &= \int \mathcal{L}(y^* \mid \lambda_t, \beta) \mathcal{L}(\lambda_t \mid \lambda_0, s_0) \mathcal{L}(d\beta, \{d\lambda_j\}, d\lambda_0, ds_0 \mid \mathbf{y}) \end{aligned}$$

So that

$$\mathcal{L}(y^*, \lambda^* \mid \mathbf{y}) = \int \mathcal{L}(y^*, \lambda^* \mid \mathbf{y}) d\lambda^*$$

We can approximate this integral with

$$\frac{1}{M} \sum_{j=1}^{M} \text{Poi}(\exp\left(x^* \beta^{(j)} + \lambda^{*(j)}\right))$$

where $\beta^{(j)}$ is the draw from the MCMC chain and $\lambda^{*(j)} \sim \mathcal{N}(\lambda_0^{(j)}, s_0^{(j)})$

```r
# All of these quantities can be computed either in stan
# (generated quantities block)
# or directly in R

predictive_poi_glm <- function(x_star, fit, ygrid) {
    betas <- as.matrix(fit$draws("beta", format = "matrix"))
    means <- exp(betas %*% x_star)
    out <- rowMeans(outer(ygrid, as.vector(means), dpois))
    return(out)
}
```

```r
predictive_poi_glmm <- function(x_star, group, fit, ygrid) {
    betas <- as.matrix(fit$draws("beta", format = "matrix"))
    lambdas <- as.matrix(fit$draws("lambda", format = "matrix"))
    means <- exp(betas %*% x_star + lambdas[, group])
    out <- rowMeans(outer(ygrid, as.vector(means), dpois))
    return(out)
}
```

```r
predictive_poi_glmm_newgroup <- function(x_star, fit, ygrid) {
    # Evaluates P(y=k | x^*, chains) for k in ygrid
    betas <- as.matrix(fit$draws("beta", format = "matrix"))
    lambda0 <- as.vector(fit$draws("lambda0", format = "matrix"))
    s0 <- as.vector(fit$draws("s0", format = "matrix"))
    lambdas <- rnorm(length(lambda0), mean = lambda0, sd = s0)
    means <- exp(betas %*% x_star + lambdas)
    out <- rowMeans(outer(ygrid, as.vector(means), dpois))
    return(out)
}
```

```r
x_star <- c(1.0, 0.5)
ygrid <- 0:14
```

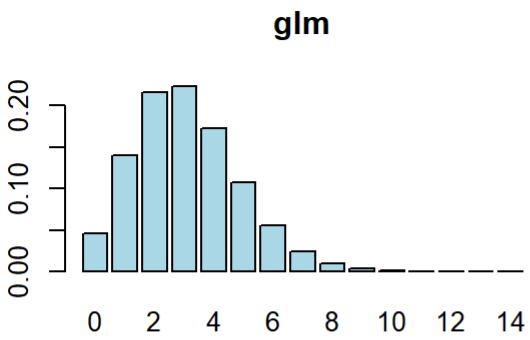
```r
par(mfrow = c(2, 2))

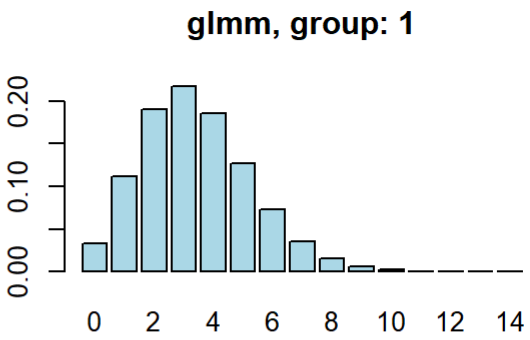
pred <- predictive_poi_glm(x_star, poi_glm_fit, ygrid)
barplot(pred, names.arg = ygrid, main = "glm", col = 'lightblue')

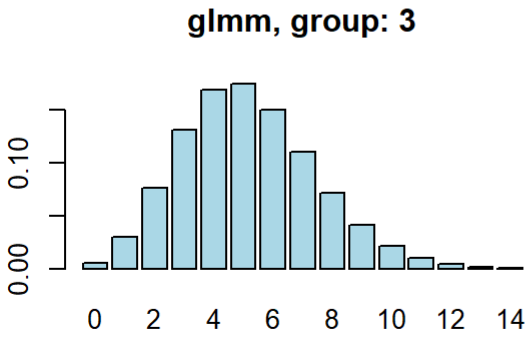
pred <- predictive_poi_glmm(c(x_star[2]), 1, poi_glmm_fit, ygrid)
barplot(pred, names.arg = ygrid, main = sprintf("glmm, group: %d", 1), col = 'lightblue')

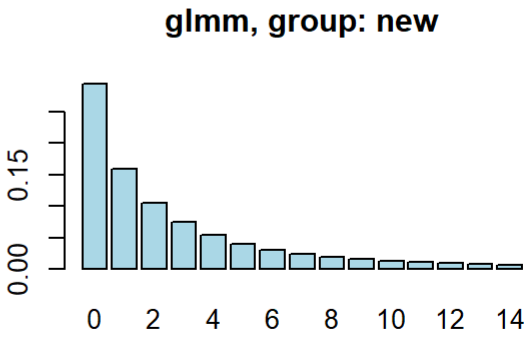
pred <- predictive_poi_glmm(c(x_star[2]), 2, poi_glmm_fit, ygrid)
barplot(pred, names.arg = ygrid, main = sprintf("glmm, group: %d", 3), col = 'lightblue')

pred <- predictive_poi_glmm_newgroup(c(x_star[2]), poi_glmm_fit, ygrid)
barplot(pred, names.arg = ygrid, main = sprintf("glmm, group: %s", "new"), col = 'lightblue')
```










```
par(mfrow = c(1, 1))
```

A more challenging example: the Ant abundance dataset

The dataset contains 60 observations: each observation measures the number of ants found on a tree.
For each tree we observe/know:

1. Trap days: how many days the tree was surveyed
2. Elevation: High (1) or Low (2)
3. Valley: three different valleys for each elevation –> nested within the elevation!
4. Number of ants

Moreover we know:

- the summer precipitation (avg)
- the summer temperature (avg)

Let $y_{i,j,r}$ the number of ants observed on tree i, at elevation j and valley r.
We assume:

$$
\begin{aligned}
y_{i,j,r} &\sim \text{Poi}(\mu_{i,j,r}) \\
\mu_{i,j,r} &= \exp(\theta_j + \alpha_{j,r} + \beta^T x_i + \gamma_j^T z_{i,j})
\end{aligned}
$$

Some terminology:

- θ_j is a random intercept $\rightarrow$ group (elevation) specific;
- $\alpha_{j,r}$ is another random intercept $\rightarrow$ group (elevation) AND subgroup (valley) specific;
- x_i form the fixed effect design matrix, β are the fixed regression coefficients;
- z_{ij} form the RANDOM effect design matrix, γ_j are the random effect regression coefficients.

In our case $x_i =$ “trap days”, $z_{ij} = (z_{ij1}, z_{ij2})$ = (average precipitation, average temperature).

```
poi_glm_rand_eff <- "
data{
    int<lower = 0> N;
    int<lower = 0> ngr;
    int<lower = 0> nsubgr;

    int<lower = 0> p_fix;
    int<lower = 0> p_ran;

    array[N] int<lower = 0> Y;
    matrix[N, p_fix] X;
    matrix[N, p_ran] Z;

    array[N] int<lower=1, upper=ngr> group;
    array[N] int<lower=1, upper=nsubgr> subgroup;
}

parameters{
    vector[p_fix] beta;            // regression coefficients (fixed)
    vector[ngr] theta;             // (group specific) random intercept
    matrix[ngr, p_ran] gamma;      // regression coefficients (random)
    vector[nsubgr] alpha;          // (group specific) random effects

    vector<lower=0>[p_fix] sigma_beta;        // variances for the prior on beta
    vector<lower=0>[ngr] sigma_theta;         // variances for the prior on theta
    matrix<lower=0>[ngr, p_ran] sigma_gamma;  // (group specific) random effects
    vector<lower=0>[nsubgr] sigma_alpha;      // (group specific) random effects
}

transformed parameters{
    vector[N] mu;
    vector[N] intercept;
    vector[N] fix_eff;
    vector[N] rand_eff;

    intercept = theta[group] + alpha[subgroup];
    fix_eff = X * beta;
    for (i in 1:N) {
        rand_eff[i] = dot_product(Z[i], gamma[group[i]]);
    }

    mu = exp(intercept + fix_eff + rand_eff);
}

model{
    for (s in 1:N) {
        Y[s] ~ poisson(mu[s]);
    }

    for (j in 1:p_fix) {
        beta[j] ~ normal(0.0, sigma_beta[j]);
        sigma_beta[j] ~ cauchy(0, 1);
    }

    for (j in 1:ngr) {
        theta[j] ~ normal(0.0, sigma_theta[j]);
        sigma_theta[j] ~ cauchy(0, 1);
    }
```

```
    for (j in 1:nsubgr){
        alpha[j] ~ normal(0.0, sigma_alpha);
        sigma_alpha[j] ~ cauchy(0, 1);
    }

    for (j in 1:p_ran) {
      for(i in 1:ngr){
        gamma[j,i] ~ normal(0.0, sigma_gamma[j,i]);
        sigma_gamma[j,i] ~ cauchy(0, 1);
      }
    }

}

generated quantities {
     vector[N] log_lik;
     for (j in 1:N){
       log_lik[j] = poisson_lpmf(Y[j] | mu[j]);
     }
}
"

# Write stan model to file
stan_file <- "./stan/poi_glmm_rand_eff.stan"
writeLines(poi_glm_rand_eff, stan_file)

# Compile stan model
poi_glmm_rand_eff <- cmdstan_model(stan_file)
```

```
ant_data <- read.csv("ant_abundance.csv")
head(ant_data)
```

```
##   year elevation     valley site trap.days num.fpodz annual.precip
## 1 2015      High East River  ER1        10         1       1150.45
## 2 2015      High East River ER10        10         0        684.24
## 3 2015      High East River  ER2        10         2       1150.45
## 4 2015      High East River  ER3        10         0       1150.45
## 5 2015      High East River  ER4        10         3       1354.83
## 6 2015      High East River  ER5        10         0       1354.83
##   summer.precip annual.temp summer.temp snowmelt.doy mound.area
## 1         67.46        2.28       10.21      141.778   0.215875
## 2         45.52        2.63       11.26      141.696   0.467075
## 3         67.46        2.28       10.21      141.655   1.895775
## 4         67.46        2.28       10.21      141.819   0.357175
## 5         67.76        1.98        9.68      142.393   0.306150
## 6         67.76        1.98        9.68      143.295   0.259050
```

```
unique(ant_data$valley)
```

```
## [1] "East River"       "Slate River"      "Washington Gulch" "Cement Creek"
## [5] "Spring Creek"     "Taylor River"
```

```
y <- ant_data$num.fpodz
X <- as.matrix(ant_data[, "trap.days", drop = FALSE])
Z <- as.matrix(ant_data[, c("summer.precip", "summer.temp")])
group <- as.integer(ant_data$elevation == "High") + 1
subgroup <- as.integer(factor(ant_data$valley))
```

```
glmm_randeff_data <- list(
    N = length(y),
    ngr = length(unique(group)),
    nsubgr = length(unique(subgroup)),
    p_fix = ncol(X),
    p_ran = ncol(Z),
    Y = y,
    X = X,
    Z = Z,
    group = group,
    subgroup = subgroup
)
```

```
poi_glmm_rand_eff_fit <- poi_glmm_rand_eff$sample(
    data = glmm_randeff_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 1000,
    refresh = 500
)
```

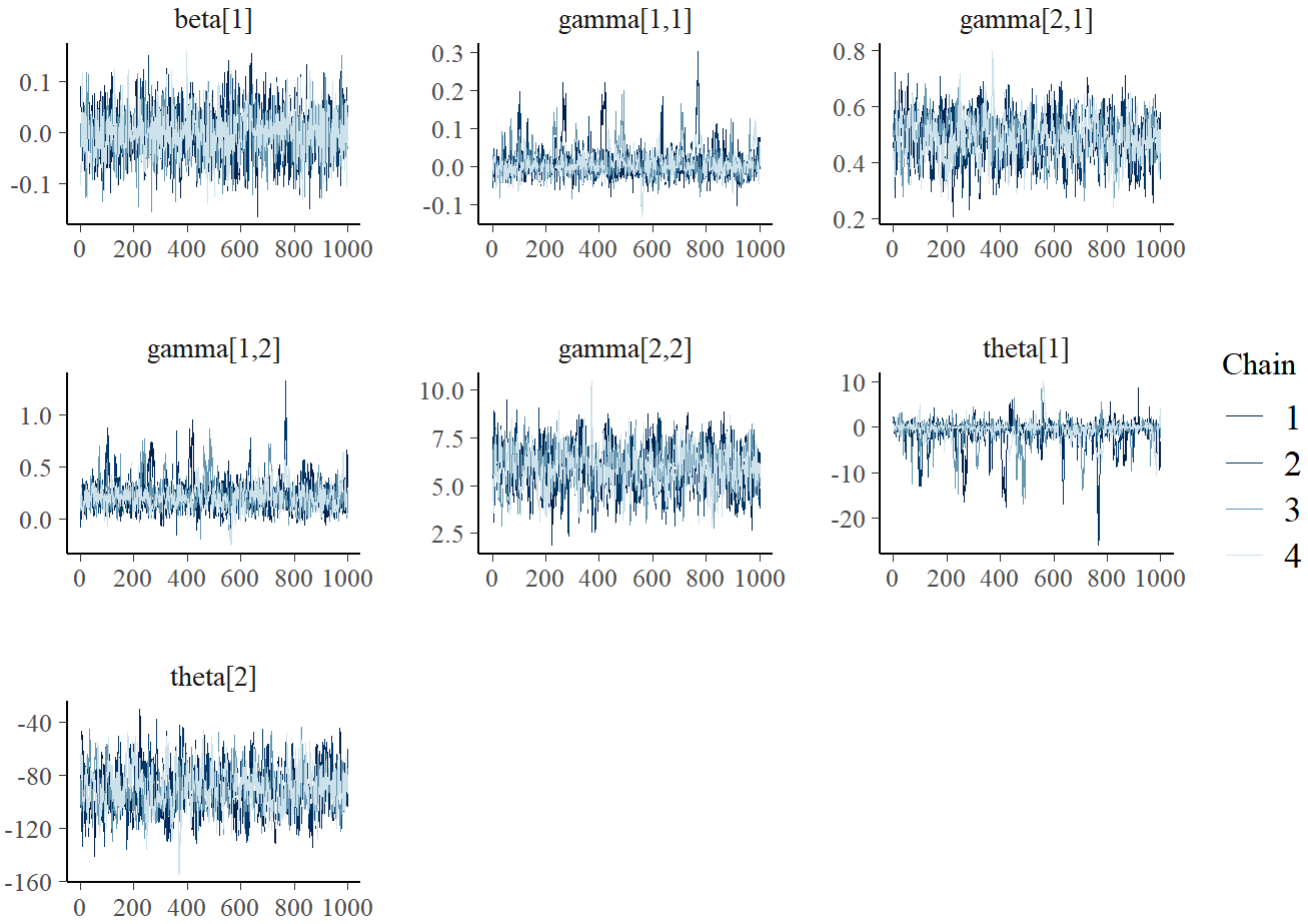
```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 44.7 seconds.
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 45.0 seconds.
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 46.1 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 48.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 46.0 seconds.
## Total execution time: 48.3 seconds.
```

```
## Warning: 13 of 4000 (0.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.
```

```
## Warning: 3929 of 4000 (98.0%) transitions hit the maximum treedepth limit of 10.
## See https://mc-stan.org/misc/warnings for details.
```

```
poi_glmm_rand_eff_data <- posterior::as_draws(poi_glmm_rand_eff_fit)
```

```
mcmc_trace(poi_glmm_rand_eff_fit$draws(c("beta", "gamma", "theta")))
```



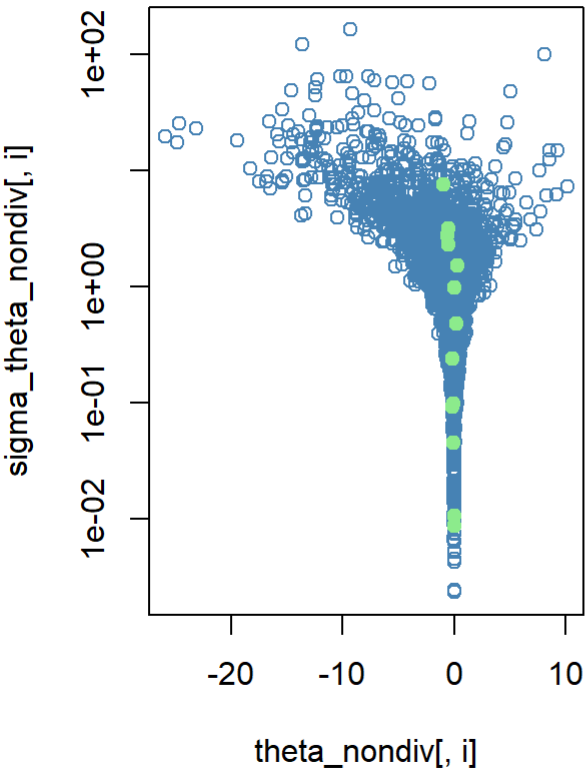

```
split_diverging <- function(fit, var_name) {
    diag_mat <- as.matrix(fit$sampler_diagnostics(format = "matrix"))
    div <- diag_mat[, "divergent__"] == 1
    vals_mat <- as.matrix(fit$draws(var_name, format = "matrix"))
    return(list(vals_mat[div, , drop = FALSE], vals_mat[!div, , drop = FALSE]))
}
```

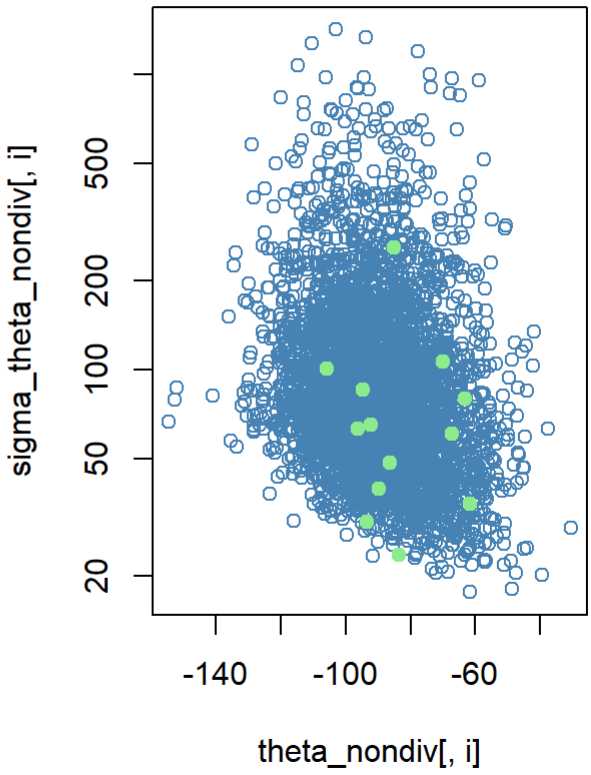
```
par(mfrow = c(1, 2))

theta_res <- split_diverging(poi_glmm_rand_eff_fit, "theta")
theta_div <- theta_res[[1]]
theta_nondiv <- theta_res[[2]]

sigma_theta_res <- split_diverging(poi_glmm_rand_eff_fit, "sigma_theta")
sigma_theta_div <- sigma_theta_res[[1]]
sigma_theta_nondiv <- sigma_theta_res[[2]]

for (i in 1:2) {
    plot(theta_nondiv[, i], sigma_theta_nondiv[, i], col = "steelblue", log = "y")
    points(theta_div[, i], sigma_theta_div[, i], col = "lightgreen", pch = 19)
}
```






```
par(mfrow = c(1, 1))
```

Let's try reparametrizing the model as follows:

$$\theta = \sigma_\theta \cdot \theta_r$$
$$\theta_r \sim \mathcal{N}(0, 1)$$
$$\sigma_\theta \sim HC(0, 1)$$

```
poi_glm_rand_eff_reparam <- "
data{
    int<lower = 0> N;
    int<lower = 0> ngr;
    int<lower = 0> nsubgr;

    int<lower = 0> p_fix;
    int<lower = 0> p_ran;

    array[N] int<lower = 0> Y;
    matrix[N, p_fix] X;
    matrix[N, p_ran] Z;

    array[N] int<lower=1, upper=ngr> group;
    array[N] int<lower=1, upper=nsubgr> subgroup;
}

parameters{
    vector[p_fix] beta;
    vector[ngr] theta_raw;
    matrix[ngr, p_ran] gamma;      // regression coefficients (random)
    vector[nsubgr] alpha;          // (group specific) random effects

    vector<lower=0>[p_fix] sigma_beta;        // variances for the prior on beta
    vector<lower=0>[ngr] sigma_theta;         // variances for the prior on theta
    matrix<lower=0>[ngr, p_ran] sigma_gamma;  // (group specific) random effects
    vector<lower=0>[nsubgr] sigma_alpha;      // (group specific) random effects
}

transformed parameters{
    vector[N] mu;
    vector[N] intercept;
    vector[N] fix_eff;
    vector[N] rand_eff;
    vector[ngr] theta;

    theta = sigma_theta .* theta_raw;

    intercept = theta[group] + alpha[subgroup];
    fix_eff = X * beta;
    for (i in 1:N) {
        rand_eff[i] = dot_product(Z[i], gamma[group[i]]);
    }

    mu = exp(intercept + fix_eff + rand_eff);
}

model{
    for (s in 1:N) {
        Y[s] ~ poisson(mu[s]);
    }

    for (j in 1:p_fix) {
        beta[j] ~ normal(0.0, sigma_beta[j]);
        sigma_beta[j] ~ cauchy(0, 1);
    }

    theta_raw ~ normal(0.0, 1.0);
    sigma_theta ~ cauchy(0, 1);
```

```
    for (j in 1:nsubgr){
        alpha[j] ~ normal(0.0, sigma_alpha);
        sigma_alpha[j] ~ cauchy(0, 1);
    }

    for (j in 1:p_ran) {
      for(i in 1:ngr){
        gamma[j,i] ~ normal(0.0, sigma_gamma[j,i]);
        sigma_gamma[j,i] ~ cauchy(0, 1);
      }
    }
}

generated quantities {
     vector[N] log_lik;
     for (j in 1:N){
           log_lik[j] = poisson_lpmf(Y[j] | mu[j]);
     }
}
"

# Write stan model to file
stan_file <- "./stan/poi_glm_rand_eff_reparam.stan"
writeLines(poi_glm_rand_eff_reparam, stan_file)

# Compile stan model
poi_glmm_rand_eff2 <- cmdstan_model(stan_file)
```

```
poi_glmm_rand_eff_fit <- poi_glmm_rand_eff2$sample(
    data = glmm_randeff_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 1000,
    refresh = 500
)
```

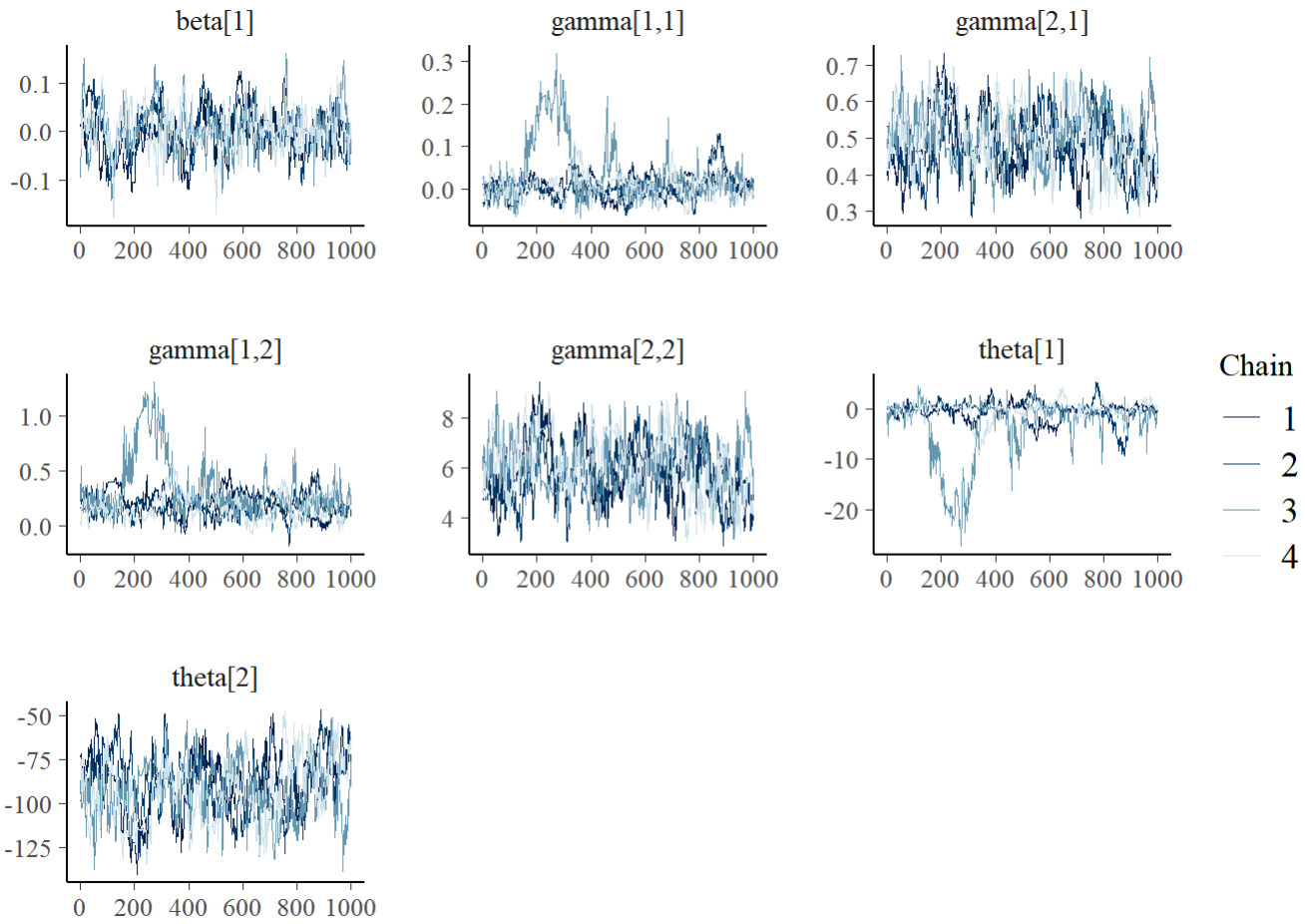
```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 41.3 seconds.
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 41.9 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 44.0 seconds.
## Chain 3 finished in 44.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 42.8 seconds.
## Total execution time: 44.1 seconds.
```

```
## Warning: 150 of 4000 (4.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.
```

```
## Warning: 3482 of 4000 (87.0%) transitions hit the maximum treedepth limit of 10.
## See https://mc-stan.org/misc/warnings for details.
```

```
poi_glmm_rand_eff_data <- posterior::as_draws(poi_glmm_rand_eff_fit)
```

```
mcmc_trace(poi_glmm_rand_eff_fit$draws(c("beta", "gamma", "theta")))
```

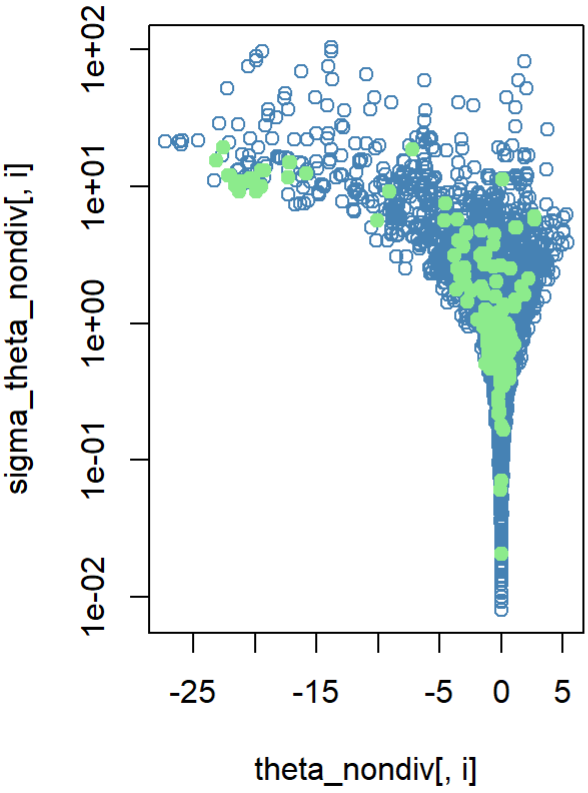


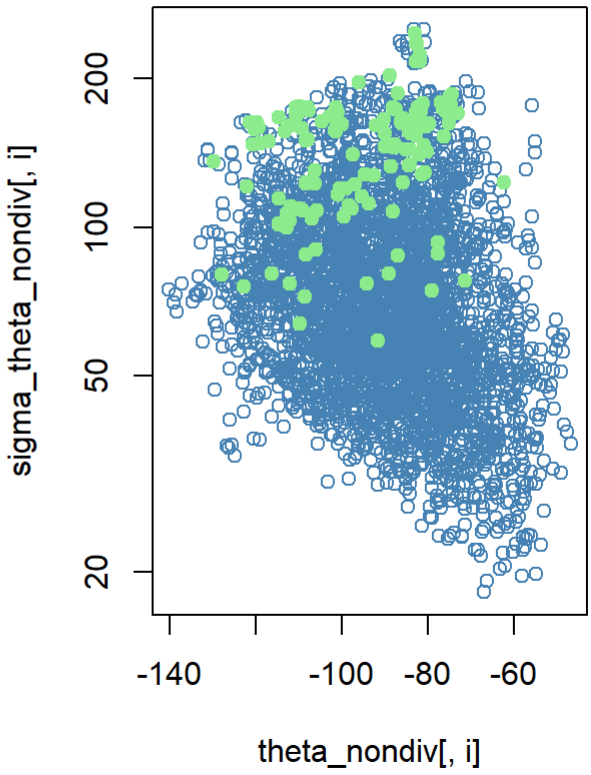

```
par(mfrow = c(1, 2))

theta_res <- split_diverging(poi_glmm_rand_eff_fit, "theta")
theta_div <- theta_res[[1]]
theta_nondiv <- theta_res[[2]]

sigma_theta_res <- split_diverging(poi_glmm_rand_eff_fit, "sigma_theta")
sigma_theta_div <- sigma_theta_res[[1]]
sigma_theta_nondiv <- sigma_theta_res[[2]]

for (i in 1:2) {
    plot(theta_nondiv[, i], sigma_theta_nondiv[, i], col = "steelblue", log = "y")
    points(theta_div[, i], sigma_theta_div[, i], col = "lightgreen", pch = 19)
}
```






```
par(mfrow = c(1, 1))
```